

Utils

```
struct BlockHeader:  
    size;  
    isUse;  
    *next;
```

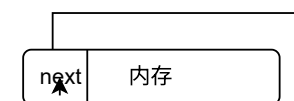
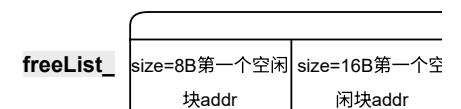
```
class SizeClass:  
    内存roundup(size)->8倍数;  
    内存大小，得到内存数后+1，
```

空闲链表，本质上不是严格意义的链表，
地址，拿到一个内存块，相当于拿到一个内存块

暴露给线程的
每个线程都有1个线程缓存对象，用于
行内存的申请与释放；
每个线程都操作自己的线程缓存对象
要锁；

内存块
内存块大小：8B，16B，24B.....2
256KB/8B = 32768种大小的内存块

每个线程缓存（对象）维护一个空
每种大小内存块对应的空闲链表；
freeListSize_，数组每一项是该大



size=8B的第一个空闲块

空闲块本身的前几个字节（BlockHeader）
空闲块的指针

链表，而是每个内存块中存有next
到了以该内存块为头节点的链表

操作

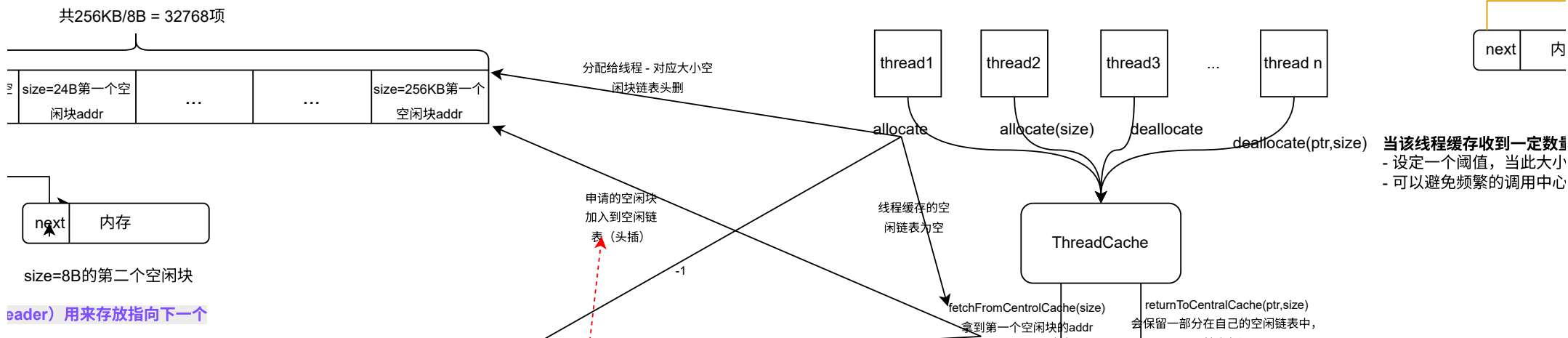
线程通过线程缓存对象进

对象，无需竞争，因此不需

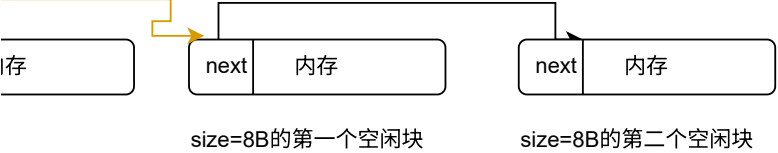
相关信息

256KB（最大），也就是说共有

空闲链表数组freeList_，数组每一项是
此外设定一个同样大小的数组
小内存块的空闲块数量；



3B为例，将归还的内存块加入
空闲链表即可（头插）



量的内存块归还时，将其打包归还给中心缓存
\的空闲块数量超过阈值时再统一一次性归还给中心缓存；
缓存；

getIndex(size)->得到空闲数组下标;

size=8B的

freeList_

size=8B的空闲块数量	size=16B的空闲块数量
---------------	----------------

所有线程共用1个中心缓存 - 需要考虑互斥

内存块相关信息

中心缓存也像线程缓存一样，维持一个空闲链表构成；但同线程缓存维持的数组不同，该数组的每一项都使用<atomic>实现无锁安全访问（互斥），<std::memory_order>提供同步顺序；

也就是说，互斥的粒度减小到了每类大小的内存块对应的链表，即可以有多线程访问唯一的中心缓存空闲链表；一个大小的内存块空闲链表 只能有1个线程访问；

实际使用时：采用自旋锁+让出CPU交替，避免忙等；

共256KB/8B = 32768项

centralFreeList_

size=8B第一个空闲块addr(atomic)	size=16B第一个空闲块addr(atomic)	size=24B第一个空闲块addr(atomic)	...
---------------------------	----------------------------	----------------------------	-----

每种大小内存块对应空闲链表的自旋锁locks_

size=8B的atomic_flag	size=16B的atomic_flag	size=24B的atomic_flag	...
---------------------	----------------------	----------------------	-----

内存块相关信息

一页固定为4KB；

给中心缓存分配时，每次至少分配8页，需求超过8页时分配（无需为8的倍数）；

因为分配是按页数分配的，所以页cache->中心cache的概念的；

维持空闲页区间链表，不同于之前的空闲块链表；空闲页区间链表按页数区分，即不同页数的页区间对应

页缓存只有1个，使用mutex控制互斥访问

每个页区间由结构体表示

size=24B的空闲块数量	...	...	size=256KB的空闲块数量
----------------	-----	-----	------------------

+1
目前实现的功能，中心缓存一次只能为线程缓存提供一个空闲块

其余归还

的数组，用
y_order>

应的空闲
数组，但

与用资源

...	size=256KB第一个空闲块addr
-----	----------------------

...	size=256KB的atomic_flag
-----	------------------------

.8页则按实际页数

iche，是不用管块

安块大小区分，空
对应不同的链表；

freeSpans_：页数->页区间

spanMap_：页地址->页区间

中心缓存取出的空闲块，到分配去的线程缓存那了

从中心缓存对应大小空闲块链表中取出该块（头删）

向页缓存请求后，需将分配到的页区间内所有空闲块加入对应链表

使用时RAII思想：采用lock_guard()保证互斥访问+自动解锁

将申请到的页区间加入链表

页缓存也没有，则向系统申请systemAlloc(numPages)

fetchRange(index)

return addr

returnRange(ptr,size,index)

接收线程缓存归还的空闲块

更新中心缓存对应大小的空闲链表

中心缓存的空闲链表该大小的空闲内存块为空

fetchFromPageCache(size)

这里会拿到一个页区间

performDelayedReturn(index)
统计该index对应空闲块来自的span，看是否有span所有块均空闲

lastReturnTime_上次归还时间

unordered_map<SpanTracker

若span所有块都空闲，归还span给页缓存

updateSpanFreeCount(tracker,newFreeBklocks,index)

从中心缓存的空闲页链表中删除对应的空闲块

归还对应span给页缓存

allocateSpan(numPages)

return addr

deallocateSpan(addr,numPages)

接收从中心缓存归还过来的Span

PageCache

页缓存是按页区间（span）分配的，所以中心缓存提供spanTracker管理收到的span，都是atomic，保证互斥

```
struct SpanTracker:
    spanAddr;
    numPages;
    blockCount;
    freeCount;
```

共256KB/8B = 32768项

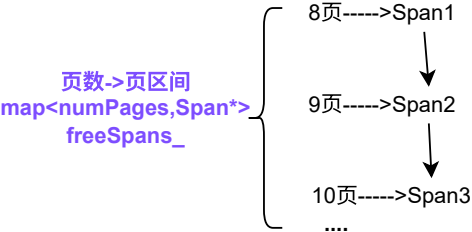
第一个空闲r(atomic)	size=16B第一个空闲块addr(atomic)	size=24B第一个空闲块addr(atomic)	...	...	size=256KB第一个空闲块addr
的空闲块上次归还时间	size=16B的空闲块上次归还时间	size=24B的空闲块上次归还时间	...	...	size=256KB的空闲块上次归还时间

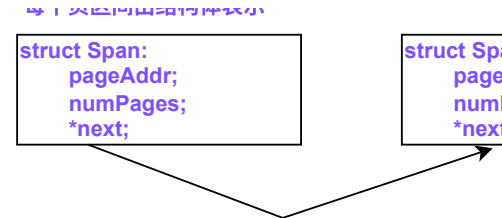
er*,size_t > spanFreeCounts span->空闲块数量的映射

3

一段连续的页区间

```
struct Span:
    pageAddr;
    numPages;
    Span* next;
```





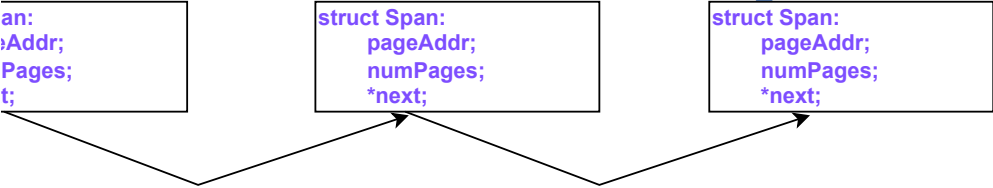
空闲页区间链表

有8页空闲块的页区间	span1	span2	span3
------------	-------	-------	-------

有9页空闲块的页区间	span1	span2	span3
------------	-------	-------	-------

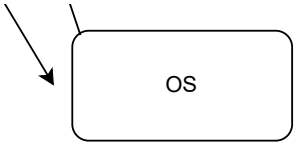
....

....



	...	...	span n	每个span的 numPages = 8
--	-----	-----	--------	-------------------------

	...	...	span n	每个span的 numPages = 9
--	-----	-----	--------	-------------------------



空闲页区间链表

有k页空闲块的页区间

nextList

span1	span2	span3
-------	-------	-------

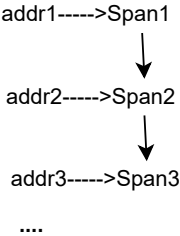
有m页空闲块的页区间

span1	span2	span3
-------	-------	-------

将回收的span更新加入
到空闲页区间链表中

..
..

页地址->页区间
map<pageAddr,Span*>
spanMap_



	...	...	span n
--	-----	-----	--------

每个span的
numPages = k

	...	...	span n
--	-----	-----	--------

每个span的
numPages = m

...
...